Lloyd k-means processes discussed for this report

k = 2 · 31,702,685 rows · same initial centres · stop when the largest centre moves by at most 1e-4

Same matrix
6 standardised fields, all rows

→

Same two centres
one shared initial file

→

Same stop rule
max shift ≤ 1e-4

Measured. Every round uses all rows, then updates the same two centres.

Python	**Workers** 1 thread, or 6 workers	**Assign** nearest centre for every row	**Update** mean of each cluster	**Repeat** until the stop rule	**Result** 2 centres
Spark CPU	**Cached RDD** local[4], [8], [12], or [*]	**Broadcast** driver sends 2 centres	**Map** nearest centre on N threads	**Reduce** sums and counts, then collect	**Repeat** 86 rounds on the full file
MapReduce	**Split** HDFS splits of the full file	**Map** nearest centre, keep every row	**Shuffle** group by cluster id	**Reduce** sum and count → new centre	**Repeat** 86 rounds, slots 4, 8, or 12
Spark GPU	**RAPIDS** RTX 3070, local[1]	**Cache** rows stay on the GPU	**Aggregate** GpuHashAggregate each round	**Repeat** same stop rule, 86 rounds	**Result** 2 centres
CUDA	**Copy** matrix onto the RTX 3070	**Kernel** one thread per row	**Update** Lloyd loop on the device	**Repeat** 86 rounds, float64	**Result** 2 centres back on the host

Not run yet. Each piece clusters on its own, then the centres are stitched.

This follows the split-then-one-reduce picture. It is not local top-N, and it is not the measured loop above.

CPU stitch	**Split** one Spark job, local[N] pieces	**Local Lloyd** each piece updates its own centres	**No round trip** no full-file shuffle each round	**Stitch** k = 2 on the local centres	**Result** 2 centres, not the global ones
GPU stitch	**Same split** pieces match the CPU run	**Local Lloyd** distance loop on the RTX 3070	**Copy back** local centres to the host	**Stitch** same stitch as the CPU path	**Result** 2 centres, compare with CPU